MINI PROJECT: END-TO-END EVENT-TO-WAREHOUSE PIPELINE

Dosen Pengampu : Isbat Uzzin Nadhori, S.Kom., M.T



Disusun untuk Memenuhi

Tugas Mata Kuliah:

Praktikum Data Processing Workflow

Oleh:

Ilham Bintang Satria

NRP. 3324600041

PROGRAM STUDI D4 SAINS DATA TERAPAN

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

GITHUB

https://github.com/YonoBengkel/dpw-miniproject

Arsitektur Pipeline

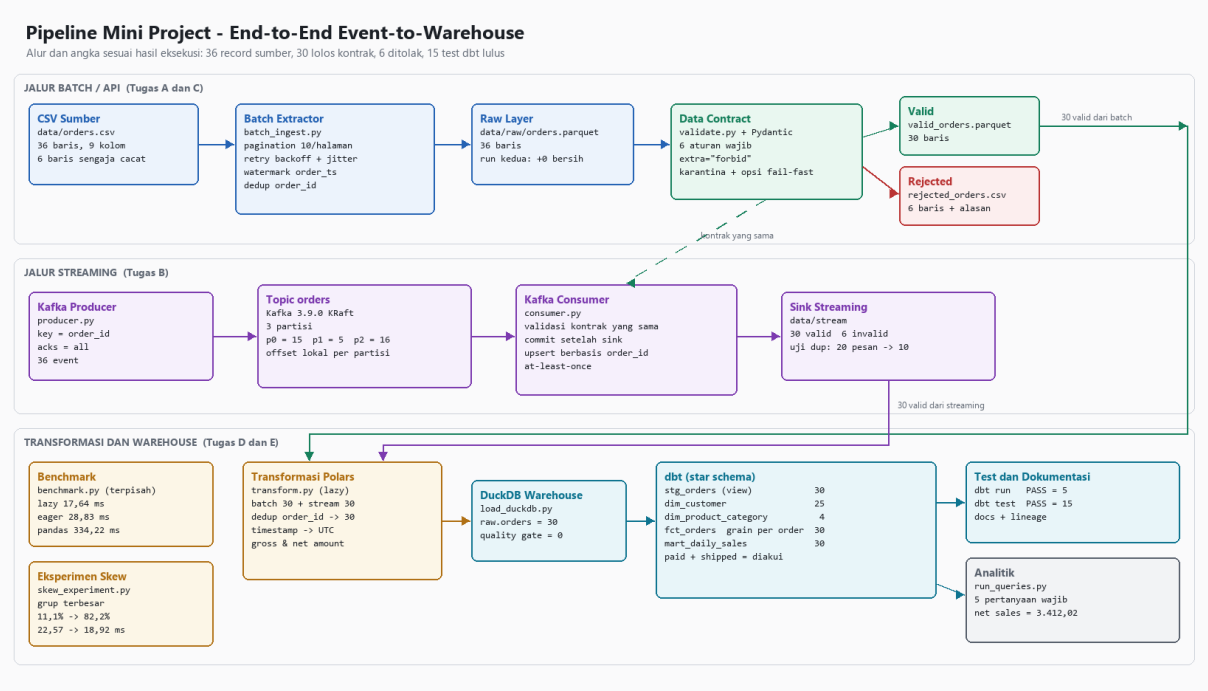


Diagram di atas memperlihatkan alur pipeline yang dibangun pada mini project ini. Jalur batch menarik 36 record dari CSV sumber secara bertahap, menyimpannya ke raw layer Parquet, lalu memisahkan 30 record valid dari 6 record yang ditolak data contract. Jalur streaming mengirim event yang sama melalui topic Kafka orders dan memvalidasinya dengan kontrak yang sama sebelum ditulis ke sink yang idempoten. Record valid dari kedua jalur kemudian digabung dan ditransformasi dengan Polars, dimuat ke warehouse DuckDB, lalu dimodelkan dengan dbt menjadi star schema yang diuji dan didokumentasikan. Benchmark Polars dan eksperimen skew dijalankan terpisah sebagai pengujian performa.

Tahap 0: Persiapan Lingkungan

A. Masuk ke working directory

```
PS D:\> cd "D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject"
PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Berpindah ke direktori kerja proyek data_processing_miniproject melalui PowerShell. Direktori ini menjadi root seluruh pekerjaan mini project, berisi dataset sumber, skrip pipeline, konfigurasi Docker Compose untuk Kafka, serta proyek dbt, sehingga seluruh path relatif pada skrip mengacu ke lokasi ini.

B. Buat virtual environment

```
PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> py -3.14 -m venv .venv
PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> .venv\Scripts\Activate.ps1
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Membuat virtual environment .venv dengan Python 3.14 lalu mengaktifkannya. Isolasi ini memisahkan dependensi mini project dari proyek lain di mesin yang sama, dan kemunculan label (.venv) pada prompt menandakan lingkungan terisolasi telah aktif.

C. Install dependensi

Memperbarui pip dari versi 26.1.1 ke 26.2.2, kemudian memasang seluruh pustaka dari requirements.txt. Pandas dan PyArrow dipakai untuk membaca CSV dan menulis Parquet, Pydantic untuk data contract, Polars untuk transformasi berperforma tinggi, kafka-python sebagai klien broker, DuckDB sebagai warehouse lokal, dan dbt-duckdb untuk pemodelan analitik.

D. Verifikasi instalasi

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python -c "import pandas, pyarrow, polars, pydantic, duckdb, kafka;
 print('pandas', pandas.__version__); print('polars', polars.__version__); print('pydantic', pydantic.__version__); print('duckdb', duckdb.__version__); pri
nt('kafka-python', kafka.__version__)"
pandas 3.0.6
polars 2.0.0
pydantic 2.14.1
duckdb 1.5.6
kafka-python 3.0.11
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> dbt --version
Core:
  - installed: 1.12.6

  The latest version of dbt-core could not be determined!
  Make sure that the following URL is accessible:
  https://pypi.org/pypi/dbt-core/json

Plugins:
  - duckdb: 1.11.0 - Up to date!


14:06:07  dbt v2 is available, and is the default for new installations. This is dbt Core v1.
          New project - install v2: https://docs.getdbt.com/docs/local/install-dbt?utm_source=dbt-cli
          Existing v1 project - upgrade guide: https://docs.getdbt.com/docs/dbt-versions/dbt-upgrade/upgrading-to-v2?utm_source=dbt-cli
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> docker --version
Docker version 29.6.2, build dfc4efb
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

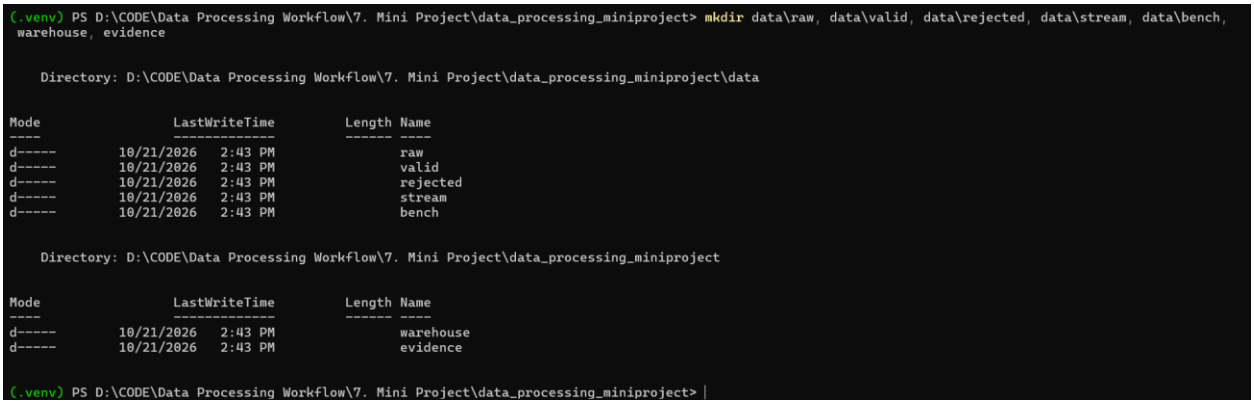
Memverifikasi versi seluruh perkakas yang terpasang, yaitu pandas 3.0.6, Polars 2.0.0, Pydantic 2.14.1, DuckDB 1.5.6, kafka-python 3.0.11, dbt-core 1.12.6 dengan adapter duckdb 1.11.0, serta Docker 29.6.2. Seluruh komponen berhasil diimpor dan adapter duckdb terdaftar pada dbt, sehingga lingkungan siap menjalankan pipeline dari ingestion sampai pemodelan.

E. Periksa dataset sumber

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python -c "import pandas as pd; df = pd.read_csv('data/orders.csv',
 dtype=str, keep_default_na=False); print(df.shape); print(df.head(5).to_string()); print(df['status'].value_counts().to_string())"
(36, 9)
  order_id customer_id             order_ts product_category quantity unit_price discount_pct payment_method     status
0    O1001        C001  2026-09-01T09:15:00Z      Electronics        1      250.0         0.05           card       paid
1    O1002        C002  2026-09-01T10:10:00Z             Home        2       45.5          0.0       transfer    shipped
2    O1003        C003  2026-09-01T11:05:00Z            Books        3      12.99          0.1        ewallet       paid
3    O1004        C001  2026-09-02T08:45:00Z      Electronics        1       89.9         0.15           card       paid
4    O1005        C004  2026-09-02T12:00:00Z          Fashion        2       39.0          0.2        ewallet  cancelled
status
paid         25
shipped       8
cancelled     1
refunded      1
pending       1
```

Memeriksa dataset data/orders.csv yang berisi 36 baris dan 9 kolom, yaitu order_id, customer_id, order_ts, product_category, quantity, unit_price, discount_pct, payment_method, dan status. Distribusi status menunjukkan paid 25, shipped 8, cancelled 1, refunded 1, dan pending 1. Status pending tersebut termasuk salah satu dari enam baris yang sengaja dibuat cacat untuk menguji data contract, sehingga tidak dihapus sebelum proses validasi.

F. Buat struktur folder kerja



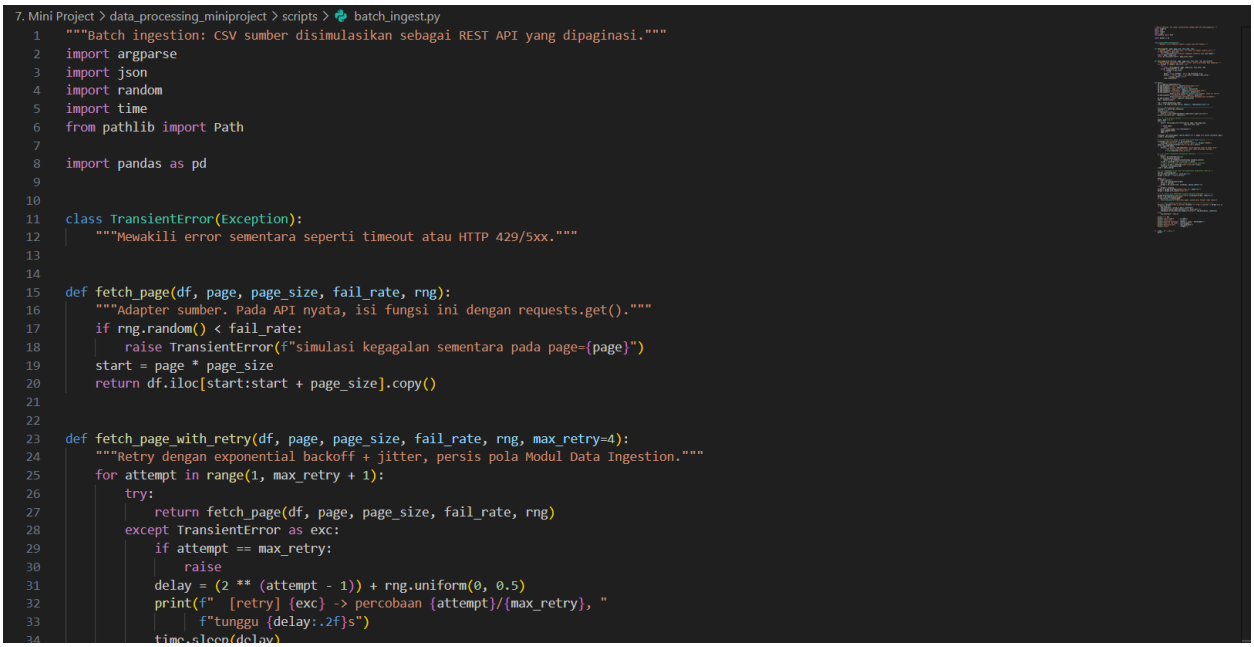
Membuat folder data/raw, data/valid, data/rejected, data/stream, data/bench, warehouse, dan evidence. Pemisahan ini menjaga setiap lapisan data tetap terpisah: raw menyimpan hasil ekstraksi apa adanya agar dapat diaudit, valid dan rejected menampung hasil validasi kontrak, sedangkan warehouse menyimpan basis data DuckDB.

Tahap 1: Batch/API Ingestion, Retry, dan Watermark (Tugas A)

A. Pahami skrip starter dan keterbatasannya

Skrip batch_ingest.py bawaan sudah memiliki kerangka pagination, penulisan Parquet, checkpoint, dan deduplikasi, tetapi memiliki empat kelemahan. Blok retry tidak pernah terpicu karena fungsi pengambil halaman tidak pernah gagal, metrik jumlah record belum dicatat, watermark ditulis tanpa memverifikasi hasil penulisan, dan baris dengan order_ts yang tidak dapat diparse berubah menjadi NaT lalu terbuang diam-diam oleh filter watermark.

B. Tulis ulang skrip batch ingestion



```python
"""Batch ingestion: CSV sumber disimulasikan sebagai REST API yang dipaginasi."""
import argparse
import json
import random
import time
from pathlib import Path

import pandas as pd


class TransientError(Exception):
    """Mewakili error sementara seperti timeout atau HTTP 429/5xx."""


def fetch_page(df, page, page_size, fail_rate, rng):
    """Adapter sumber. Pada API nyata, isi fungsi ini dengan requests.get()."""
    if rng.random() < fail_rate:
        raise TransientError(f"simulasi kegagalan sementara pada page={page}")
    start = page * page_size
    return df.iloc[start:start + page_size].copy()


def fetch_page_with_retry(df, page, page_size, fail_rate, rng, max_retry=4):
    """Retry dengan exponential backoff + jitter, persis pola Modul Data Ingestion."""
    for attempt in range(1, max_retry + 1):
        try:
            return fetch_page(df, page, page_size, fail_rate, rng)
        except TransientError as exc:
            if attempt == max_retry:
                raise
            delay = (2 ** (attempt - 1)) + rng.uniform(0, 0.5)
            print(f"  [retry] {exc} -> percobaan {attempt}/{max_retry}, "
                  f"tunggu {delay:.2f}s")
            time.sleep(delay)
```

Menulis ulang skrip dengan menambahkan kelas TransientError dan fungsi fetch_page_with_retry yang menerapkan exponential backoff disertai jitter. Urutan siklusnya disusun menjadi baca watermark, tarik seluruh halaman, normalisasi waktu, filter inkremental, gabungkan dan deduplikasi berdasarkan order_id, tulis Parquet, verifikasi hasil tulis, baru memperbarui watermark. Penempatan watermark di langkah terakhir memastikan kegagalan penulisan tidak pernah membuat data terlewat.

C. Run pertama (full load)

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/batch_ingest.py --source data/orders.csv --out data/
raw --page-size 10
watermark awal : None
page=0 rows=10
page=1 rows=10
page=2 rows=10
page=3 rows=6
  [catatan] 1 record memiliki order_ts tidak valid dan tetap diteruskan ke raw layer untuk divalidasi kontrak: ['O1034']
------------------------------------------------------------
record dibaca      : 36
record baru        : 36
duplikat diabaikan : 0
total di raw layer : 36
watermark akhir    : 2026-09-11T15:00:00+00:00
output             : data\raw\orders.parquet
```

Menjalankan ekstraksi pertama dengan ukuran halaman 10 record. Skrip membaca empat halaman berisi 10, 10, 10, dan 6 record, sehingga seluruh 36 record masuk ke data/raw/orders.parquet dan watermark tersimpan pada 2026-09-11T15:00:00+00:00. Record O1034 yang memiliki order_ts tidak valid tetap diteruskan ke raw layer agar ditolak secara eksplisit oleh data contract, bukan hilang di tahap ingestion.

D. Run kedua: bukti incremental tidak menggandakan data

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/batch_ingest.py --source data/orders.csv --out data/
raw --page-size 10
watermark awal : 2026-09-11T15:00:00+00:00
page=0 rows=10
page=1 rows=10
page=2 rows=10
page=3 rows=6
  [catatan] 1 record memiliki order_ts tidak valid dan tetap diteruskan ke raw layer untuk divalidasi kontrak: ['O1034']
------------------------------------------------------------
record dibaca      : 36
record baru        : 1
duplikat diabaikan : 1
total di raw layer : 36
watermark akhir    : 2026-09-11T15:00:00+00:00
output             : data\raw\orders.parquet
```

Metrik	Run Pertama	Run Kedua
Record dibaca	36	36
Record baru	36	1
Duplikat diabaikan	0	1
Total di raw layer	36	36
Watermark akhir	2026-09-11T15:00:00+00:00	2026-09-11T15:00:00+00:00

Run kedua membaca watermark tersimpan lalu kembali menarik seluruh 36 record dari sumber. Hanya satu record yang lolos filter watermark, yaitu O1034 yang timestamp-nya rusak, dan record tersebut langsung dikenali sebagai duplikat oleh deduplikasi berbasis order_id. Total raw layer tetap 36 dan tidak berlipat menjadi 72, membuktikan pipeline dapat dijalankan ulang tanpa menggandakan data.

E. Uji retry dengan kegagalan yang disuntikkan

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/batch_ingest.py --source data/orders.csv --out data/
raw --fail-rate 0.5
watermark awal : 2026-09-11T15:00:00+00:00
page=0 rows=10
  [retry] simulasi kegagalan sementara pada page=1 -> percobaan 1/4, tunggu 1.14s
  [retry] simulasi kegagalan sementara pada page=1 -> percobaan 2/4, tunggu 2.37s
page=1 rows=10
page=2 rows=10
  [retry] simulasi kegagalan sementara pada page=3 -> percobaan 1/4, tunggu 1.21s
  [retry] simulasi kegagalan sementara pada page=3 -> percobaan 2/4, tunggu 2.11s
page=3 rows=6
  [retry] simulasi kegagalan sementara pada page=4 -> percobaan 1/4, tunggu 1.10s
  [catatan] 1 record memiliki order_ts tidak valid dan tetap diteruskan ke raw layer untuk divalidasi kontrak: ['O1034']
------------------------------------------------------------
record dibaca      : 36
record baru        : 1
duplikat diabaikan : 1
total di raw layer : 36
watermark akhir    : 2026-09-11T15:00:00+00:00
output             : data\raw\orders.parquet
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Menyuntikkan kegagalan sementara dengan peluang 50 persen per halaman. Halaman 1 dan 3 masing-masing gagal dua kali sebelum berhasil, dengan jeda yang menaik dari sekitar 1,1 detik menjadi 2,1 hingga 2,4 detik, sesuai pola exponential backoff 1, 2, dan 4 detik yang ditambah jitter acak. Hasil akhirnya tetap identik dengan run kedua, yaitu total 36 record. Pada REST API sungguhan, retry hanya pantas untuk error sementara seperti timeout, 429, 500, 502, 503, dan 504, sedangkan 400, 401, 403, dan 404 tidak boleh diulang secara buta karena pengulangannya tidak akan pernah berhasil.

Tahap 2: Data Contract dan Fail-Fast (Tugas C)

A. Definisikan kontrak data

7. Mini Project > data_processing_miniproject > scripts > contract.py > ...

```python
"""Kontrak data untuk satu record order."""
from datetime import datetime
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field


class Order(BaseModel):
    model_config = ConfigDict(extra="forbid")

    order_id: str = Field(min_length=1)
    customer_id: str = Field(min_length=1)
    order_ts: datetime
    product_category: str = Field(min_length=1)
    quantity: int = Field(gt=0)
    unit_price: float = Field(ge=0)
    discount_pct: float = Field(ge=0, le=1)
    payment_method: str = Field(min_length=1)
    status: Literal["paid", "shipped", "cancelled", "refunded"]
```

Mendefinisikan kontrak Order dengan Pydantic. Enam aturan wajib diterapkan: order_id dan customer_id tidak boleh kosong, quantity harus bilangan bulat lebih dari nol, unit_price tidak boleh negatif, discount_pct berada pada rentang 0 sampai 1, order_ts harus dapat diparse menjadi timestamp, dan status hanya boleh paid, shipped, cancelled, atau refunded. Opsi extra="forbid" menolak kolom yang tidak dikenal untuk mendeteksi schema drift, sedangkan mode strict sengaja tidak dipakai karena sumbernya CSV sehingga seluruh nilai terbaca sebagai teks dan memerlukan koersi tipe.

B. Bangun skrip validasi batch

7. Mini Project > data_processing_miniproject > scripts > validate.py > ...

```python
"""Validasi raw layer terhadap kontrak: pisahkan record valid dan rejected."""
import argparse
import json
from pathlib import Path

import pandas as pd
from pydantic import ValidationError

from contract import Order


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default="data/raw/orders.parquet")
    ap.add_argument("--valid-out", default="data/valid/valid_orders.parquet")
    ap.add_argument("--rejected-out", default="data/rejected/rejected_orders.csv")
    ap.add_argument("--fail-fast", action="store_true",
                    help="hentikan batch pada pelanggaran pertama")
    args = ap.parse_args()

    df = pd.read_parquet(args.source)
    records = df.astype(object).where(pd.notna(df), None).to_dict("records")

    valid_rows, rejected_rows = [], []
```

Membangun validate.py yang membaca raw layer, memvalidasi setiap record secara terpisah, lalu memisahkan hasilnya. Record valid disimpan ke data/valid/valid_orders.parquet, sedangkan record yang gagal disimpan ke data/rejected/rejected_orders.csv beserta order_id, alasan penolakan, dan payload aslinya agar dapat diaudit. Opsi --fail-fast disediakan untuk menghentikan batch pada pelanggaran pertama.

C. Jalankan validasi

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/validate.py
total record   : 36
valid          : 30
rejected       : 6
tingkat validitas: 83.33%

  O1031: quantity: Input should be greater than 0
  O1032: unit_price: Input should be greater than or equal to 0
  O1033: discount_pct: Input should be less than or equal to 1
  O1034: order_ts: Input should be a valid datetime or date, invalid character in year
  O1035: status: Input should be 'paid', 'shipped', 'cancelled' or 'refunded'
  O1036: customer_id: String should have at least 1 character
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

order_id	Aturan yang Dilanggar	Pesan Error
O1031	quantity harus lebih dari nol	Input should be greater than 0
O1032	unit_price tidak boleh negatif	Input should be greater than or equal to 0
O1033	discount_pct harus 0 sampai 1	Input should be less than or equal to 1
O1034	order_ts harus timestamp valid	Input should be a valid datetime or date
O1035	status harus salah satu nilai sah	Input should be 'paid', 'shipped', 'cancelled' or 'refunded'
O1036	customer_id wajib diisi	String should have at least 1 character

Validasi terhadap 36 record menghasilkan 30 record valid dan 6 record ditolak, atau tingkat validitas 83,33 persen. Keenam record yang ditolak masing-masing melanggar tepat satu aturan berbeda, dan seluruh alasannya tercatat jelas sehingga dapat ditindaklanjuti. Record cacat berhenti di lapisan ini dan tidak pernah diteruskan ke warehouse.

D. Demonstrasikan mode fail-fast

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/validate.py --fail-fast
[FAIL-FAST] batch dihentikan pada O1031: quantity: Input should be greater than 0
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Menjalankan validasi dengan opsi --fail-fast, sehingga batch langsung dihentikan pada pelanggaran pertama, yaitu O1031 dengan quantity bernilai nol. Mode fail-fast cocok untuk kiriman yang harus utuh atau tidak sama sekali, sedangkan karantina per-record lebih cocok untuk data operasional harian yang cacatnya sporadis. Pipeline ini memakai karantina per-record sebagai mode utama agar 30 record yang sah tetap diproses, dengan fail-fast sebagai opsi yang terbukti berfungsi.

Tahap 3: Event-Driven Streaming dengan Kafka (Tugas B)

A. Jalankan broker

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> docker compose up -d
[+] up 2/2
 ✔Network data_processing_miniproject_default Created                                                                0.1s
 ✔Container dpw-kafka                         Started                                                                1.2s
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> docker compose ps
NAME        IMAGE                COMMAND                  SERVICE   CREATED         STATUS                            PORTS
dpw-kafka   apache/kafka:3.9.0   "/__cacert_entrypoin…"   kafka     9 seconds ago   Up 9 seconds (health: starting)   0.0.0.0:9092->9092/tcp, [::]:9092->909
2/tcp
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Menjalankan broker Apache Kafka 3.9.0 melalui Docker Compose. Broker berjalan dalam mode KRaft yang mengelola metadata klaster sendiri melalui protokol Raft tanpa ZooKeeper. Container dpw-kafka berstatus Up dan memetakan port 9092 sehingga dapat diakses oleh producer dan consumer dari mesin lokal.

B. Buat topic orders

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --create -
-topic orders --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
Created topic orders.
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --describe
 --topic orders --bootstrap-server localhost:9092
Topic: orders   TopicId: YKl1bGlHT--qwcP5kELKUw PartitionCount: 3       ReplicationFactor: 1    Configs:
        Topic: orders   Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: orders   Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: orders   Partition: 2    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
```

Membuat topic orders dengan tiga partisi dan replication factor satu. Tiga partisi memberi ruang paralelisme hingga tiga consumer dalam satu group, sedangkan replication factor satu sudah memadai untuk broker tunggal di lingkungan praktikum. Hasil describe menunjukkan ketiga partisi memiliki leader pada broker 1 dengan in-sync replica yang lengkap.

C. Bangun producer

7. Mini Project > data_processing_miniproject > scripts > producer.py

```python
"""Menerbitkan event order dari CSV ke topic Kafka."""
import argparse
import json

import pandas as pd
from kafka import KafkaProducer


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default="data/orders.csv")
    ap.add_argument("--topic", default="orders")
    ap.add_argument("--limit", type=int, default=36)
    ap.add_argument("--duplicate", type=int, default=0,
                    help="kirim ulang N event pertama untuk uji idempotensi")
    args = ap.parse_args()

    producer = KafkaProducer(
        bootstrap_servers="localhost:9092",
        key_serializer=lambda k: k.encode("utf-8"),
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
        acks="all",
    )

    df = pd.read_csv(args.source, dtype=str, keep_default_na=False)
    events = df.head(args.limit).to_dict("records")
    if args.duplicate:
        events += events[:args.duplicate]

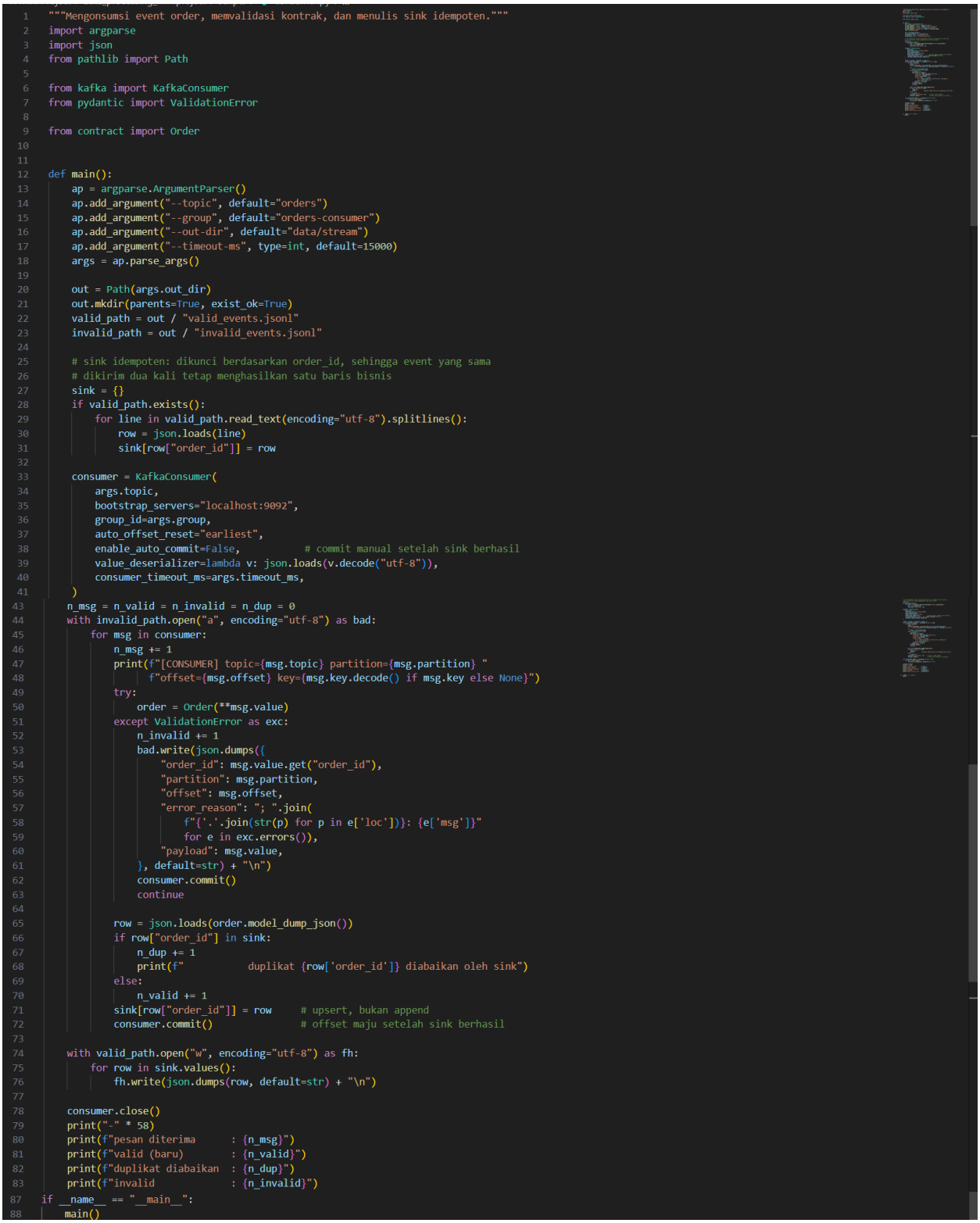
    for event in events:
        # key = order_id agar seluruh event satu order masuk ke partisi yang sama
        future = producer.send(args.topic, key=event["order_id"], value=event)
        meta = future.get(timeout=10)
        print(f"[PRODUCER] {event['order_id']} -> partition={meta.partition} "
              f"offset={meta.offset}")

    producer.flush()
    print(f"[PRODUCER] selesai, {len(events)} event terkirim "
          f"({args.duplicate} di antaranya duplikat sengaja)")


if __name__ == "__main__":
    main()
```

Producer membaca record dari data/orders.csv lalu mengirimnya sebagai JSON ke topic orders dengan order_id sebagai message key dan pengaturan acks all. Hash dari message key menentukan partisi tujuan, sehingga seluruh event dengan order_id yang sama selalu jatuh ke partisi yang sama dan urutannya terjaga. Opsi --duplicate disediakan untuk mengirim ulang sebagian event guna menguji idempotensi.

D. Bangun consumer



```python
"""Mengonsumsi event order, memvalidasi kontrak, dan menulis sink idempoten."""
import argparse
import json
from pathlib import Path

from kafka import KafkaConsumer
from pydantic import ValidationError

from contract import Order


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--topic", default="orders")
    ap.add_argument("--group", default="orders-consumer")
    ap.add_argument("--out-dir", default="data/stream")
    ap.add_argument("--timeout-ms", type=int, default=15000)
    args = ap.parse_args()

    out = Path(args.out_dir)
    out.mkdir(parents=True, exist_ok=True)
    valid_path = out / "valid_events.jsonl"
    invalid_path = out / "invalid_events.jsonl"

    # sink idempoten: dikunci berdasarkan order_id, sehingga event yang sama
    # dikirim dua kali tetap menghasilkan satu baris bisnis
    sink = {}
    if valid_path.exists():
        for line in valid_path.read_text(encoding="utf-8").splitlines():
            row = json.loads(line)
            sink[row["order_id"]] = row

    consumer = KafkaConsumer(
        args.topic,
        bootstrap_servers="localhost:9092",
        group_id=args.group,
        auto_offset_reset="earliest",
        enable_auto_commit=False,           # commit manual setelah sink berhasil
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
        consumer_timeout_ms=args.timeout_ms,
    )
    n_msg = n_valid = n_invalid = n_dup = 0
    with invalid_path.open("a", encoding="utf-8") as bad:
        for msg in consumer:
            n_msg += 1
            print(f"[CONSUMER] topic={msg.topic} partition={msg.partition} "
                  f"offset={msg.offset} key={msg.key.decode() if msg.key else None}")
            try:
                order = Order(**msg.value)
            except ValidationError as exc:
                n_invalid += 1
                bad.write(json.dumps({
                    "order_id": msg.value.get("order_id"),
                    "partition": msg.partition,
                    "offset": msg.offset,
                    "error_reason": "; ".join(
                        f"{'.'.join(str(p) for p in e['loc'])}: {e['msg']}"
                        for e in exc.errors()),
                    "payload": msg.value,
                }, default=str) + "\n")
                consumer.commit()
                continue

            row = json.loads(order.model_dump_json())
            if row["order_id"] in sink:
                n_dup += 1
                print(f"           duplikat {row['order_id']} diabaikan oleh sink")
            else:
                n_valid += 1
            sink[row["order_id"]] = row     # upsert, bukan append
            consumer.commit()               # offset maju setelah sink berhasil

    with valid_path.open("w", encoding="utf-8") as fh:
        for row in sink.values():
            fh.write(json.dumps(row, default=str) + "\n")

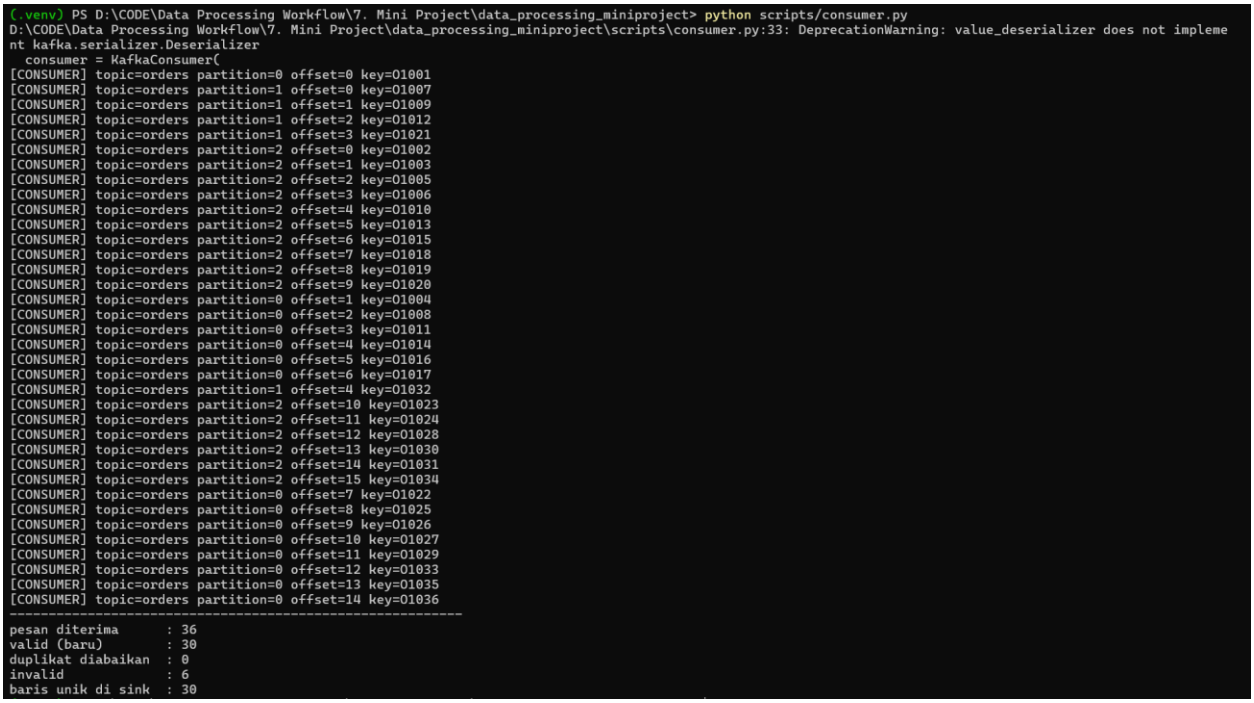
    consumer.close()
    print("-" * 58)
    print(f"pesan diterima      : {n_msg}")
    print(f"valid (baru)        : {n_valid}")
    print(f"duplikat diabaikan  : {n_dup}")
    print(f"invalid             : {n_invalid}")
if __name__ == "__main__":
    main()
```

Consumer berlangganan ke topic orders dalam satu consumer group, membaca dari offset paling awal, dan mematikan auto commit. Setiap pesan divalidasi dengan kontrak yang sama dengan jalur batch, lalu dicatat topic, partition, dan offset-nya. Record valid ditulis ke sink yang dikunci

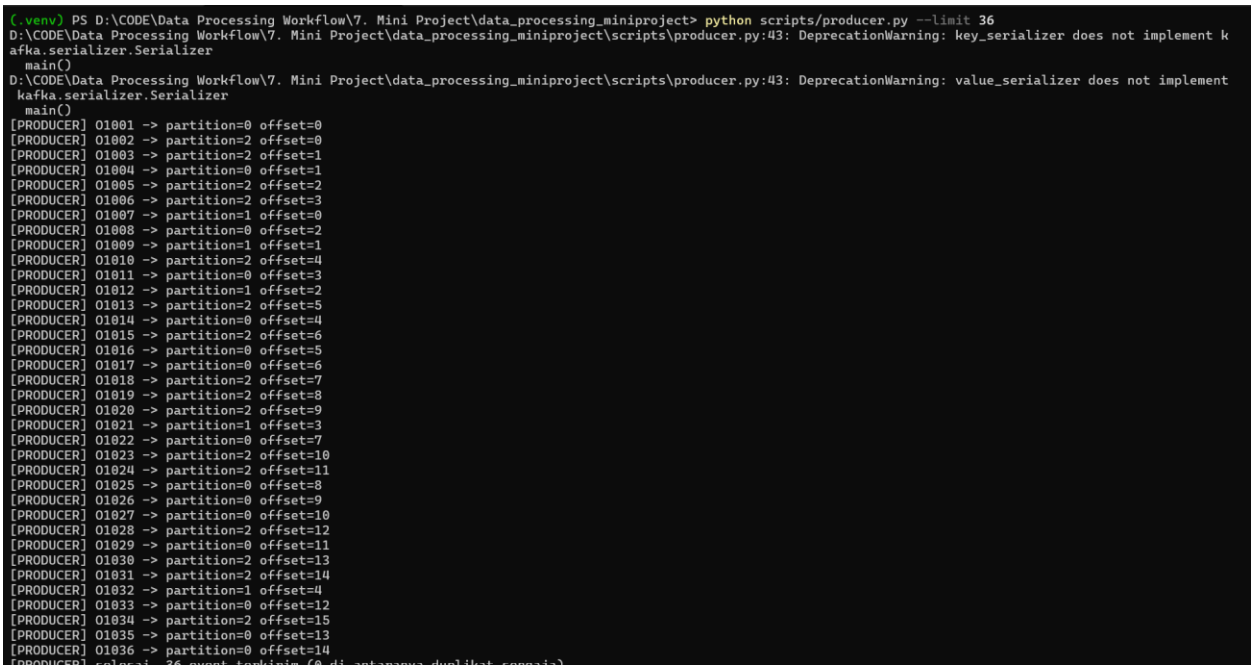
berdasarkan order_id sehingga bersifat upsert, record tidak valid disimpan beserta alasannya, dan offset baru di-commit setelah sink berhasil ditulis.

E. Uji kasus normal

Terminal 1



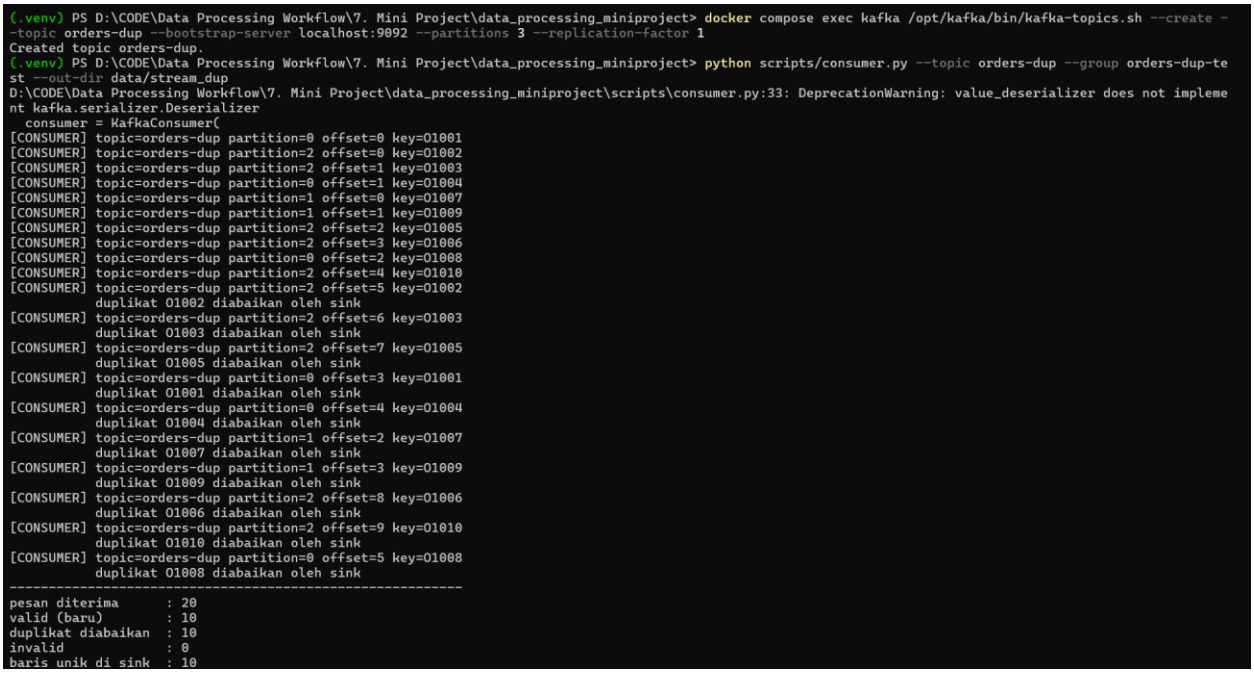
Terminal 2



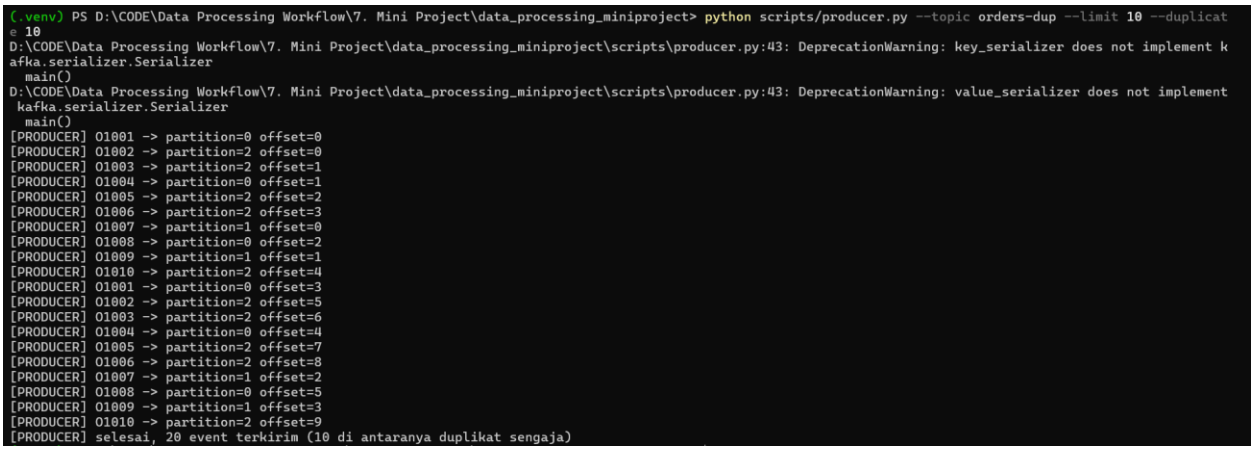
Producer mengirim 36 event, dan consumer menerima seluruhnya dengan hasil 30 valid, 6 invalid, dan 30 baris unik di sink, sama persis dengan hasil jalur batch. Event tersebar tidak merata ke tiga partisi, yaitu 15 event di partisi 0, 5 event di partisi 1, dan 16 event di partisi 2, karena penempatannya ditentukan hash message key dan bukan dibagi rata. Offset dimulai dari nol pada masing-masing partisi, menunjukkan offset bersifat lokal per partisi, sedangkan consumer group menyimpan posisi pembacaan terakhirnya di broker.

F. Uji duplikasi dan idempotensi sink

Terminal 1



Terminal 2



Membuat topic orders-dup tersendiri lalu mengirim sepuluh event pertama dua kali. Consumer menerima 20 pesan, tetapi hanya 10 yang tercatat sebagai baris baru, 10 sisanya dikenali sebagai duplikat, dan sink tetap berisi 10 baris. Setiap event duplikat juga jatuh ke partisi yang sama

dengan aslinya, misalnya O1002 pada partisi 2 offset 0 dan offset 5, membuktikan penempatan berbasis key bersifat deterministik. Upsert berbasis order_id inilah yang mencegah duplikasi pada level bisnis.

G. Pernyataan batas jaminan pengiriman

Implementasi ini memberikan jaminan at-least-once: offset di-commit setelah sink berhasil ditulis, sehingga bila consumer berhenti sebelum commit, pesan akan diproses ulang dan tidak hilang. Duplikasi yang timbul dari pemrosesan ulang tersebut dinetralkan pada level bisnis melalui upsert berbasis order_id. Exactly-once end-to-end tidak diklaim, karena transaksi Kafka hanya menjamin alur Kafka ke Kafka dan tidak otomatis membuat penulisan ke berkas atau DuckDB menjadi atomik.

H. Matikan broker setelah selesai



Menghentikan dan menghapus container dpw-kafka beserta network-nya dengan docker compose down setelah seluruh pengujian streaming selesai, agar tidak ada layanan yang tertinggal berjalan dan menghabiskan sumber daya.

Tahap 4: Transformasi dan Optimasi Performa (Tugas D)

A. Rumus metrik gross dan net amount

Metrik transaksi dihitung dengan rumus gross_amount sama dengan quantity dikali unit_price, dan net_amount sama dengan gross_amount dikali satu dikurangi discount_pct. Nilai discount_pct 0,10 berarti diskon sepuluh persen, bukan 0,10 persen. Rumus ini dihitung satu kali pada transformasi Polars, lalu hasilnya diteruskan apa adanya oleh model dbt sehingga angka antar-lapisan tidak berbeda.

B. Transformasi gabungan batch dan streaming

Transform.py:

7. Mini Project > data_processing_miniproject > scripts > transform.py

```python
"""Transformasi Polars: gabungkan record valid dari jalur batch dan streaming.

Kedua jalur sudah divalidasi kontrak yang sama. Skrip ini menyeragamkan tipe data,
membuang duplikat per order_id, menghitung gross_amount dan net_amount, lalu menulis
hasilnya sebagai Parquet yang dimuat ke warehouse oleh load_duckdb.py.
"""
import argparse
from pathlib import Path

import polars as pl

KOLOM = ["order_id", "customer_id", "order_ts", "product_category", "quantity",
         "unit_price", "discount_pct", "payment_method", "status"]


def seragamkan(lf: pl.LazyFrame, sumber: str) -> pl.LazyFrame:
    """Samakan tipe kolom agar batch (datetime) dan streaming (teks) bisa digabung."""
    ts = pl.col("order_ts")
    if lf.collect_schema()["order_ts"] == pl.String:
        ts = ts.str.to_datetime(time_unit="us", time_zone="UTC")
    return lf.select(KOLOM).with_columns(
        ts.dt.convert_time_zone("UTC").alias("order_ts"),
        pl.col("quantity").cast(pl.Int64),
        pl.col("unit_price").cast(pl.Float64),
        pl.col("discount_pct").cast(pl.Float64),
        pl.lit(sumber).alias("sumber"),
    )


def transform(batch: pl.LazyFrame, stream: pl.LazyFrame | None) -> pl.LazyFrame:
    """Gabung, dedup per order_id (versi streaming menang karena datang belakangan),
    lalu hitung metrik. Semuanya lazy; eksekusi terjadi saat collect()."""
    bagian = [seragamkan(batch, "batch")]
    if stream is not None:
        bagian.append(seragamkan(stream, "stream"))
    return (
        pl.concat(bagian, how="vertical")
        .unique(subset="order_id", keep="last", maintain_order=True)
        .with_columns(
            (pl.col("quantity") * pl.col("unit_price")).alias("gross_amount"),
        )
        .with_columns(
            (pl.col("gross_amount") * (1 - pl.col("discount_pct"))).alias("net_amount"),
            pl.col("order_ts").dt.date().alias("order_date"),
        )
        .sort("order_id")
    )


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--batch", default="data/valid/valid_orders.parquet")
    ap.add_argument("--stream", default="data/stream/valid_events.jsonl")
    ap.add_argument("--out", default="data/transformed/orders_transformed.parquet")
    args = ap.parse_args()

    batch = pl.scan_parquet(args.batch)
    stream_path = Path(args.stream)
    punya_stream = stream_path.exists() and stream_path.stat().st_size > 0
    stream = pl.scan_ndjson(stream_path) if punya_stream else None
    if not punya_stream:
        print(f"[catatan] {stream_path} tidak ada atau kosong; hanya jalur batch yang dipakai")

    n_batch = batch.select(pl.len()).collect().item()
    n_stream = stream.select(pl.len()).collect().item() if stream is not None else 0

    hasil = transform(batch, stream).collect()

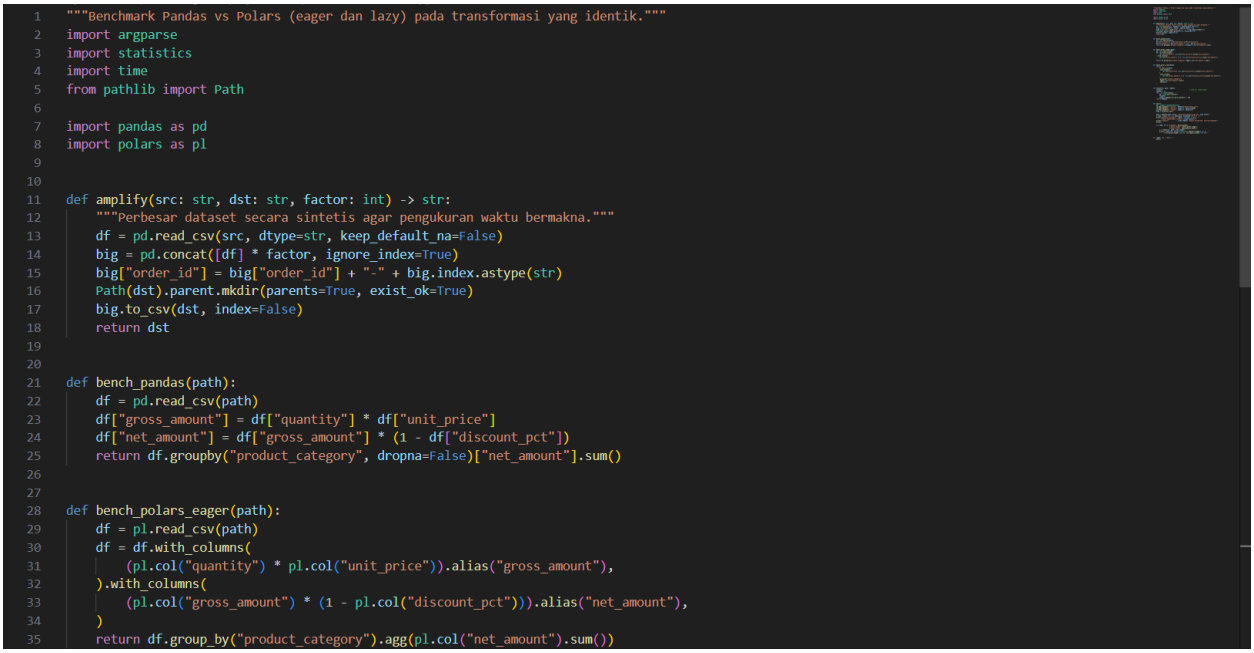
    Path(args.out).parent.mkdir(parents=True, exist_ok=True)
    hasil.write_parquet(args.out)

    print(f"record valid dari batch    : {n_batch}")
    print(f"record valid dari streaming: {n_stream}")
    print(f"gabungan sebelum dedup     : {n_batch + n_stream}")
    print(f"setelah dedup per order_id : {hasil.height}")
    print(f"total net_amount           : {hasil['net_amount'].sum():.2f}")
    print(f"output                     : {args.out}")
    print()
    # dicetak lewat pandas: tabel Polars memakai karakter garis kotak yang gagal
    # ditampilkan oleh konsol Windows berencoding cp1252
    contoh = hasil.select("order_id", "order_ts", "quantity", "unit_price",
                          "discount_pct", "gross_amount", "net_amount", "sumber").head(5)
    print(contoh.to_pandas().to_string(index=False))


if __name__ == "__main__":
    main()
```

Menjalankan transform.py yang menggabungkan record valid dari jalur batch dan jalur streaming dengan Polars lazy. Sebanyak 30 record dari batch dan 30 record dari streaming digabung menjadi 60 baris, lalu duplikat dibuang berdasarkan order_id sehingga tersisa 30 order, karena order yang sama masuk melalui kedua jalur. Kolom order_ts dari streaming yang masih berupa teks diseragamkan menjadi timestamp UTC, kemudian gross_amount, net_amount, dan order_date dihitung, dengan total net_amount 3.948,47 untuk seluruh status. Seluruh langkah disusun sebagai rencana lazy melalui scan_parquet dan scan_ndjson lalu dieksekusi sekali saat collect(), dan hasilnya ditulis ke data/transformed/orders_transformed.parquet. Tes pada tests/test_transform.py memastikan dedup, perhitungan diskon, dan normalisasi UTC berjalan benar.

C. Benchmark Pandas dan Polars



```python
"""Benchmark Pandas vs Polars (eager dan lazy) pada transformasi yang identik."""
import argparse
import statistics
import time
from pathlib import Path

import pandas as pd
import polars as pl


def amplify(src: str, dst: str, factor: int) -> str:
    """Perbesar dataset secara sintetis agar pengukuran waktu bermakna."""
    df = pd.read_csv(src, dtype=str, keep_default_na=False)
    big = pd.concat([df] * factor, ignore_index=True)
    big["order_id"] = big["order_id"] + "-" + big.index.astype(str)
    Path(dst).parent.mkdir(parents=True, exist_ok=True)
    big.to_csv(dst, index=False)
    return dst


def bench_pandas(path):
    df = pd.read_csv(path)
    df["gross_amount"] = df["quantity"] * df["unit_price"]
    df["net_amount"] = df["gross_amount"] * (1 - df["discount_pct"])
    return df.groupby("product_category", dropna=False)["net_amount"].sum()


def bench_polars_eager(path):
    df = pl.read_csv(path)
    df = df.with_columns(
        (pl.col("quantity") * pl.col("unit_price")).alias("gross_amount"),
    ).with_columns(
        (pl.col("gross_amount") * (1 - pl.col("discount_pct"))).alias("net_amount"),
    )
    return df.group_by("product_category").agg(pl.col("net_amount").sum())
```

```python
    return (
        pl.scan_csv(path)
        .with_columns(
            (pl.col("quantity") * pl.col("unit_price")).alias("gross_amount"),
        )
        .with_columns(
            (pl.col("gross_amount") * (1 - pl.col("discount_pct"))).alias("net_amount"),
        )
        .group_by("product_category")
        .agg(pl.col("net_amount").sum())
        .collect()
    )


def timeit(fn, path, repeat):
    fn(path)                                    # warm-up, tidak diukur
    samples = []
    for _ in range(repeat):
        t0 = time.perf_counter()
        fn(path)
        samples.append(time.perf_counter() - t0)
    return samples


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default="data/orders.csv")
    ap.add_argument("--factor", type=int, default=5000)
    ap.add_argument("--repeat", type=int, default=7)
    args = ap.parse_args()

    path = amplify(args.source, "data/bench/orders_big.csv", args.factor)
    n_rows = sum(1 for _ in open(path, encoding="utf-8")) - 1
    print(f"dataset benchmark : {path} ({n_rows:,} baris, "
          f"hasil amplifikasi sintetis x{args.factor})")
    print(f"repetisi          : {args.repeat} (median dilaporkan, warm-up dibuang)")
    print()
```

```python
                    ("polars-eager", bench_polars_eager),
                    ("polars-lazy", bench_polars_lazy)]:
        s = timeit(fn, path, args.repeat)
        print(f"{name:14s} median={statistics.median(s)*1000:8.2f} ms  "
              f"min={min(s)*1000:8.2f} ms  max={max(s)*1000:8.2f} ms")


if __name__ == "__main__":
    main()
```

```
dataset benchmark : data/bench/orders_big.csv (180,000 baris, hasil amplifikasi sintetis x5000)
repetisi          : 7 (median dilaporkan, warm-up dibuang)

pandas         median=  334.22 ms  min=  326.96 ms  max=  343.01 ms
polars-eager   median=   28.83 ms  min=   26.40 ms  max=   45.07 ms
polars-lazy    median=   17.64 ms  min=   17.22 ms  max=   21.36 ms
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Pendekatan	Median (ms)	Min (ms)	Maks (ms)
Pandas	334,22	326,96	343,01
Polars eager	28,83	26,40	45,07
Polars lazy	17,64	17,22	21,36

Membandingkan Pandas, Polars eager, dan Polars lazy pada transformasi yang identik, yaitu menghitung gross dan net amount lalu mengagregasi per kategori. Dataset 36 baris diperbesar secara sintetis 5.000 kali menjadi 180.000 baris, setiap pendekatan diulang tujuh kali, dan median dilaporkan setelah warm-up dibuang. Polars lazy mencatat median 17,64 ms, Polars eager 28,83 ms, dan Pandas 334,22 ms. Selisih sebesar ini sebagian besar berasal dari pembacaan CSV: Polars memakai pembaca multi-thread, dan mode lazy hanya membaca kolom yang dibutuhkan berkat projection pushdown, sedangkan Pandas memuat seluruh kolom. Hasil ini berlaku untuk mesin dan beban kerja ini, bukan klaim bahwa Polars selalu lebih cepat.

D. Eksperimen data skew

```python
"""Eksperimen data skew: satu customer_id mendominasi distribusi grup."""
import argparse
import statistics
import time
from pathlib import Path

import polars as pl
import pandas as pd


def build_datasets(src, out_dir, factor, skew_ratio):
    df = pd.read_csv(src, dtype=str, keep_default_na=False)
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)

    uniform = pd.concat([df] * factor, ignore_index=True)
    uniform["order_id"] = uniform["order_id"] + "-" + uniform.index.astype(str)
    uniform_path = out / "orders_uniform.csv"
    uniform.to_csv(uniform_path, index=False)

    skewed = uniform.copy()
    n_hot = int(len(skewed) * skew_ratio)
    skewed.loc[:n_hot, "customer_id"] = "C001"      # satu pelanggan dominan
    skewed_path = out / "orders_skewed.csv"
    skewed.to_csv(skewed_path, index=False)
    return uniform_path, skewed_path


def group_by_customer(path):
    return (
        pl.scan_csv(path)
        .with_columns(
            (pl.col("quantity") * pl.col("unit_price")
             * (1 - pl.col("discount_pct"))).alias("net_amount"))
        .group_by("customer_id")
        .agg(pl.col("net_amount").sum(), pl.len().alias("n_rows"))
        .collect()
    )
    group_by_customer(path)
    s = []
    for _ in range(repeat):
        t0 = time.perf_counter()
        group_by_customer(path)
        s.append(time.perf_counter() - t0)
    return statistics.median(s)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default="data/orders.csv")
    ap.add_argument("--out-dir", default="data/bench")
    ap.add_argument("--factor", type=int, default=5000)
    ap.add_argument("--skew-ratio", type=float, default=0.8)
    ap.add_argument("--repeat", type=int, default=5)
    args = ap.parse_args()

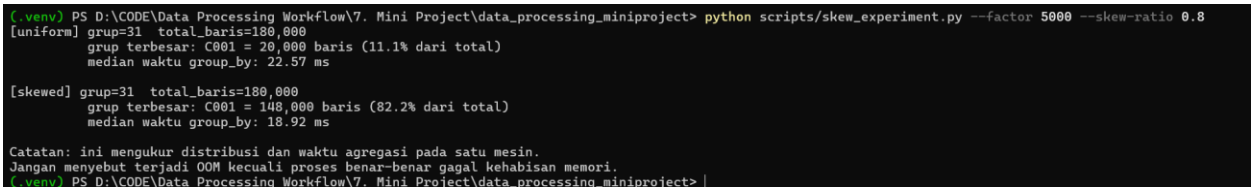
    uniform_path, skewed_path = build_datasets(
        args.source, args.out_dir, args.factor, args.skew_ratio)

    for label, path in [("uniform", uniform_path), ("skewed", skewed_path)]:
        result = group_by_customer(path).sort("n_rows", descending=True)
        top = result.row(0, named=True)
        total = result["n_rows"].sum()
        print(f"[{label}] grup={result.height}  total_baris={total:,}")
        print(f"          grup terbesar: {top['customer_id']} "
              f"= {top['n_rows']:,} baris ({top['n_rows']/total*100:.1f}% dari total)")
        print(f"          median waktu group_by: {timeit(path, args.repeat)*1000:.2f} ms")
        print()

    print("Catatan: ini mengukur distribusi dan waktu agregasi pada satu mesin.")
    print("Jangan menyebut terjadi OOM kecuali proses benar-benar gagal kehabisan memori.")


if __name__ == "__main__":
    main()
```

Jalankan:



Skenario	Jumlah Grup	Grup Terbesar	Porsi	Median (ms)
Uniform	31	C001 (20.000 baris)	11,1%	22,57
Skewed	31	C001 (148.000 baris)	82,2%	18,92

Membuat dua dataset berukuran sama, 180.000 baris, yaitu dataset seragam dan dataset skewed di mana 80 persen baris dipaksa menjadi milik customer C001. Grup terbesar naik dari 20.000 baris atau 11,1 persen menjadi 148.000 baris atau 82,2 persen dari total. Waktu group_by justru tidak melambat, yaitu 22,57 ms pada data seragam dan 18,92 ms pada data skewed, karena di satu mesin total baris yang diproses tetap sama dan agregasi berbasis hash cukup menumpuk hasil pada satu kunci. Skew baru menjadi masalah serius ketika kunci dipetakan ke worker atau partisi berbeda, sebab satu worker kebagian sebagian besar data sementara yang lain menganggur. Mitigasinya antara lain agregasi parsial, salting key, pemrosesan per batch, proyeksi kolom, dan menghindari materialisasi yang tidak perlu. Tidak terjadi kegagalan kehabisan memori selama eksperimen.

Tahap 5: Warehouse DuckDB dan Pemodelan dbt (Tugas E)

A. Muat hasil transformasi ke DuckDB

7. Mini Project > data_processing_miniproject > scripts > load_duckdb.py > ...

```python
"""Memuat hasil transformasi Polars (record valid batch + streaming) ke DuckDB."""
import argparse
from pathlib import Path

import duckdb


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default="data/transformed/orders_transformed.parquet")
    ap.add_argument("--db", default="warehouse/warehouse.duckdb")
    args = ap.parse_args()

    Path(args.db).parent.mkdir(parents=True, exist_ok=True)
    con = duckdb.connect(args.db)
    con.execute("CREATE SCHEMA IF NOT EXISTS raw;")
    con.execute(f"""
        CREATE OR REPLACE TABLE raw.orders AS
        SELECT * FROM read_parquet('{Path(args.source).as_posix()}');
    """)

    n = con.execute("SELECT COUNT(*) FROM raw.orders").fetchone()[0]
    print(f"raw.orders dimuat: {n} baris")
    print(con.execute("""
        SELECT status, COUNT(*) AS n
        FROM raw.orders GROUP BY status ORDER BY n DESC
    """).fetchdf().to_string(index=False))

    # bukti bahwa data cacat tidak masuk warehouse
    bad = con.execute("""
        SELECT COUNT(*) FROM raw.orders
        WHERE quantity <= 0 OR unit_price < 0
           OR discount_pct < 0 OR discount_pct > 1
           OR customer_id IS NULL OR customer_id = ''
           OR status NOT IN ('paid','shipped','cancelled','refunded')
    """).fetchone()[0]
    print(f"\nrecord melanggar kontrak di warehouse: {bad} (harus 0)")
    con.close()


if __name__ == "__main__":
    main()
```

Jalankan

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/load_duckdb.py
raw.orders dimuat: 30 baris
   status  n
     paid 20
  shipped  8
cancelled  1
 refunded  1

record melanggar kontrak di warehouse: 0 (harus 0)
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject>
```

Memuat data/transformed/orders_transformed.parquet hasil transformasi Polars ke tabel raw.orders di warehouse DuckDB. Sebanyak 30 baris berhasil dimuat dengan rincian paid 20, shipped 8, cancelled 1, dan refunded 1, dan tabel ini sudah membawa kolom gross_amount, net_amount, serta order_date. Pemeriksaan quality gate terhadap seluruh aturan kontrak menghasilkan nol record pelanggar, membuktikan data cacat tidak pernah sampai ke warehouse.

B. Siapkan struktur proyek dbt

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> mkdir dbt_project\models\staging, dbt_project\models\marts


    Directory: D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project\models


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/22/2026   3:20 PM                staging
d-----        10/22/2026   3:20 PM                marts
```

dbt_project.yml

7. Mini Project > data_processing_miniproject > dbt_project > Y dbt_project.yml

```yaml
name: mini_project
version: '1.0.0'
config-version: 2
profile: mini_project
model-paths: [models]
test-paths: [tests]
models:
  mini_project:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

profiles.yml

7. Mini Project > data_processing_miniproject > dbt_project > Y profiles.yml

```yaml
mini_project:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: D:/CODE/Data Processing Workflow/7. Mini Project/data_processing_miniproject/warehouse/warehouse.duckdb
      threads: 4
```

Membuat folder model staging dan marts beserta berkas konfigurasinya. Berkas dbt_project.yml menetapkan lapisan staging sebagai view yang ringan dan lapisan marts sebagai table yang cepat dibaca, sedangkan profiles.yml menghubungkan dbt dengan berkas warehouse.duckdb melalui adapter duckdb.

C. Bangun model staging

7. Mini Project > data_processing_miniproject > dbt_project > models > staging > stg_orders.sql

```sql
with source as (
    select * from {{ source('raw', 'orders') }}
)

select
    cast(order_id         as varchar)   as order_id,
    cast(customer_id      as varchar)   as customer_id,
    cast(order_ts         as timestamp) as order_ts,
    cast(order_date       as date)      as order_date,
    cast(product_category as varchar)   as product_category,
    cast(payment_method   as varchar)   as payment_method,
    cast(status           as varchar)   as status,
    cast(quantity         as integer)   as quantity,
    cast(unit_price       as double)    as unit_price,
    cast(discount_pct     as double)    as discount_pct,
    -- gross_amount dan net_amount sudah dihitung oleh scripts/transform.py (Polars)
    cast(gross_amount     as double)    as gross_amount,
    cast(net_amount       as double)    as net_amount
from source
```

Model stg_orders membaca source raw.orders lalu menyeragamkan tipe setiap kolom dan meneruskan gross_amount, net_amount, serta order_date yang sudah dihitung oleh transformasi Polars, sehingga perhitungan metrik hanya dilakukan di satu tempat. Seluruh model hilir mengambil data dari lapisan ini.

D. Bangun model dimensi, fakta, dan mart

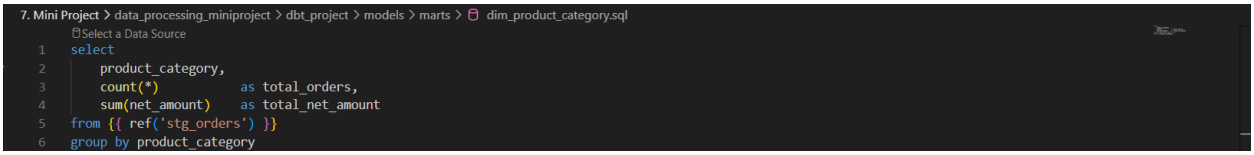
dim_customer.sql

7. Mini Project > data_processing_miniproject > dbt_project > models > marts > dim_customer.sql

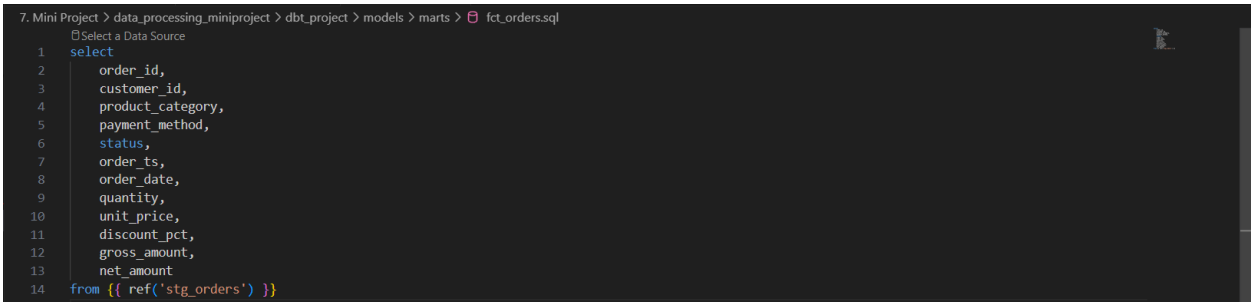
Select a Data Source

```sql
select
    customer_id,
    min(order_ts)  as first_order_ts,
    max(order_ts)  as last_order_ts,
    count(*)       as total_orders
from {{ ref('stg_orders') }}
group by customer_id
```

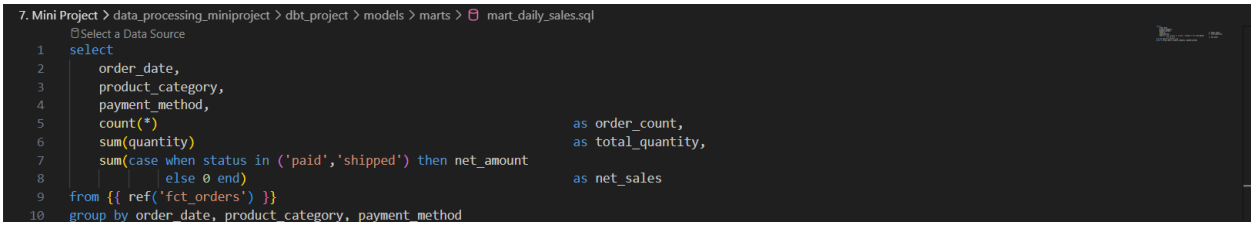
dim_product_category.sql



fct_orders.sql



mart_daily_sales.sql



Membangun dim_customer dengan grain satu baris per customer_id, dim_product_category dengan grain satu baris per kategori, fct_orders dengan grain satu baris per order_id, serta mart_daily_sales yang mengagregasi penjualan per tanggal, kategori, dan metode pembayaran. Desain ini mengikuti star schema karena fact terhubung langsung ke dimensi yang relatif denormalisasi, sehingga join tetap sedikit dan query analitik sederhana. Mart hanya mengakui status paid dan shipped sebagai penjualan bersih.

E. Tambahkan pengujian dan dokumentasi

7. Mini Project > data_processing_miniproject > dbt_project > models > Y schema.yml

```yaml
version: 2

sources:
  - name: raw
    schema: raw
    description: "Record order yang telah lolos data contract."
    tables:
      - name: orders

models:
  - name: stg_orders
    description: "Order bersih dengan tipe terstandardisasi dan metrik turunan."
    columns:
      - name: order_id
        description: "Identitas bisnis order."
        tests: [unique, not_null]
      - name: net_amount
        description: "gross_amount dikali (1 - discount_pct)."
        tests: [not_null]

  - name: dim_customer
    description: "Dimensi pelanggan, satu baris per customer_id."
    columns:
      - name: customer_id
        tests: [unique, not_null]

  - name: dim_product_category
    description: "Dimensi kategori produk, satu baris per kategori."
    columns:
      - name: product_category
        tests: [unique, not_null]

  - name: fct_orders
    description: "Fakta order dengan grain satu baris per order_id."
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_customer')
                field: customer_id
      - name: product_category
        tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_product_category')
                field: product_category
      - name: status
        tests:
          - accepted_values:
              arguments:
                values: ['paid', 'shipped', 'cancelled', 'refunded']

  - name: mart_daily_sales
    description: "Agregasi penjualan harian per kategori dan metode pembayaran."
    columns:
      - name: order_date
        tests: [not_null]
```

Menambahkan schema.yml berisi deskripsi model dan 15 pengujian. Test unique dan not_null dipasang pada kunci setiap model untuk menjamin tidak ada duplikasi maupun nilai kosong, test relationships memastikan setiap customer_id dan product_category di fct_orders benar-benar ada di dimensinya, dan test accepted_values membatasi status pada empat nilai yang sah. Argumen test ditulis di bawah kunci arguments agar sesuai sintaks dbt 1.12 tanpa peringatan deprecation.

F. Jalankan dbt

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> cd dbt_project
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project> dbt debug
09:46:21  Running with dbt=1.12.6
09:46:21  dbt version: 1.12.6
09:46:21  python version: 3.14.5
09:46:21  python path: D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\.venv\Scripts\python.exe
09:46:21  os info: Windows-11-10.0.26200-SP0
09:46:21  Using profiles dir at D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project
09:46:21  Using profiles.yml file at D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project\profiles.yml
09:46:21  Using dbt_project.yml file at D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project\dbt_project.yml
09:46:21  adapter type: duckdb
09:46:21  adapter version: 1.11.0
09:46:22  Configuration:
09:46:22    profiles.yml file [OK found and valid]
09:46:22    dbt_project.yml file [OK found and valid]
09:46:22  Required dependencies:
09:46:22   - git [OK found]

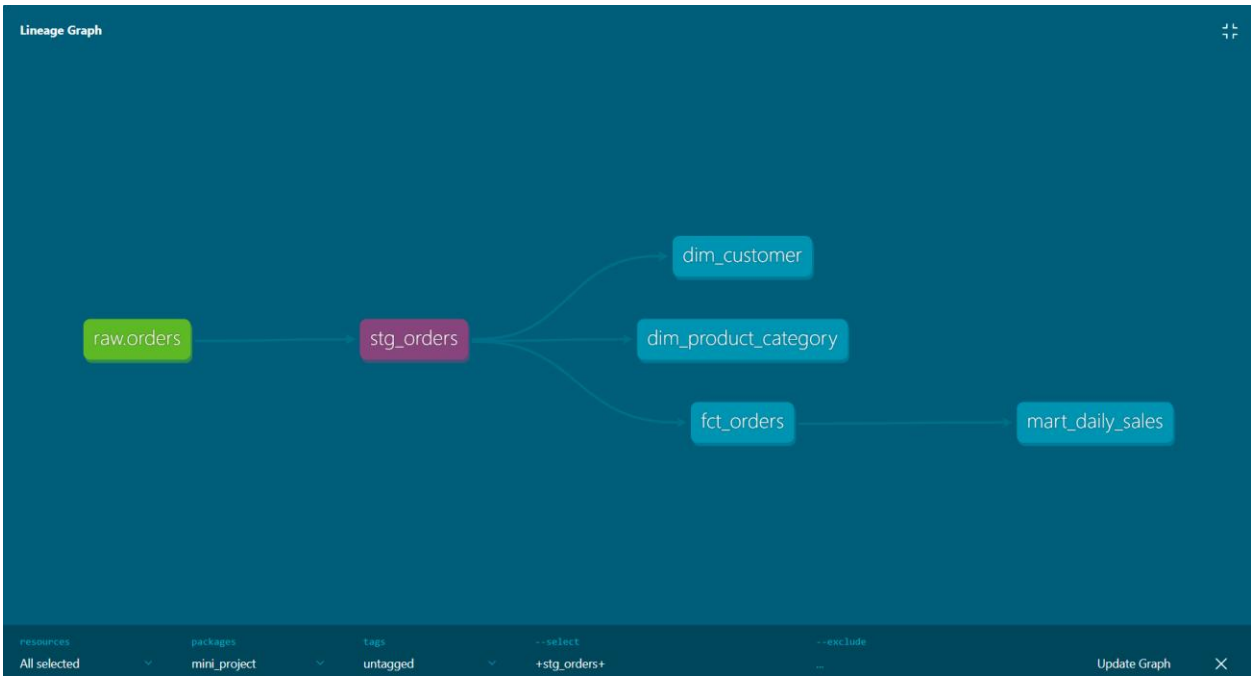
09:46:22  Connection:
09:46:22    database: warehouse
09:46:22    schema: main
09:46:22    path: D:/CODE/Data Processing Workflow/7. Mini Project/data_processing_miniproject/warehouse/warehouse.duckdb
09:46:22    config_options: None
09:46:22    extensions: None
09:46:22    settings: {}
09:46:22    external_root: .
09:46:22    use_credential_provider: None
09:46:22    attach: None
09:46:22    filesystems: None
09:46:22    remote: None
09:46:22    plugins: None
09:46:22    disable_transactions: False
09:46:22  Registered adapter: duckdb=1.11.0
09:46:26    Connection test: [OK connection ok]

09:46:26  All checks passed!
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project> dbt run
09:46:35  Running with dbt=1.12.6
09:46:35  Registered adapter: duckdb=1.11.0
09:46:36  Unable to do partial parsing because saved manifest not found. Starting full parse.
09:46:39  Found 5 models, 15 data tests, 1 source, 500 macros
09:46:39
09:46:39  Concurrency: 4 threads (target='dev')
09:46:39
09:46:39  1 of 5 START sql view model main.stg_orders .................................... [RUN]
09:46:39  1 of 5 OK created sql view model main.stg_orders ............................... [OK in 0.13s]
09:46:39  3 of 5 START sql table model main.dim_product_category ......................... [RUN]
09:46:39  4 of 5 START sql table model main.fct_orders ................................... [RUN]
09:46:39  2 of 5 START sql table model main.dim_customer ................................. [RUN]
09:46:40  3 of 5 OK created sql table model main.dim_product_category .................... [OK in 0.24s]
09:46:40  4 of 5 OK created sql table model main.fct_orders .............................. [OK in 0.24s]
09:46:40  2 of 5 OK created sql table model main.dim_customer ............................ [OK in 0.24s]
09:46:40  5 of 5 START sql table model main.mart_daily_sales ............................. [RUN]
09:46:40  5 of 5 OK created sql table model main.mart_daily_sales ........................ [OK in 0.05s]
09:46:40
09:46:40  Finished running 4 table models, 1 view model in 0 hours 0 minutes and 0.74 seconds (0.74s).
09:46:40
09:46:40  Completed successfully
09:46:40
09:46:40  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=5
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project> dbt test
09:46:47  Running with dbt=1.12.6
09:46:47  Registered adapter: duckdb=1.11.0
09:46:49  Found 5 models, 15 data tests, 1 source, 500 macros
09:46:49
09:46:49  Concurrency: 4 threads (target='dev')
09:46:49
09:46:49  3 of 15 START test not_null_dim_product_category_product_category .............. [RUN]
09:46:49  2 of 15 START test not_null_dim_customer_customer_id ........................... [RUN]
09:46:49  1 of 15 START test accepted_values_fct_orders_status__paid__shipped__cancelled__refunded  [RUN]
09:46:49  4 of 15 START test not_null_fct_orders_customer_id ............................. [RUN]
09:46:49  2 of 15 PASS not_null_dim_customer_customer_id ................................. [PASS in 0.13s]
09:46:49  3 of 15 PASS not_null_dim_product_category_product_category .................... [PASS in 0.15s]
09:46:49  1 of 15 PASS accepted_values_fct_orders_status__paid__shipped__cancelled__refunded  [PASS in 0.12s]
09:46:49  5 of 15 START test not_null_fct_orders_order_id ............................... [RUN]
09:46:49  6 of 15 START test not_null_fct_orders_product_category ....................... [RUN]
09:46:49  7 of 15 START test not_null_mart_daily_sales_order_date ....................... [RUN]
09:46:49  4 of 15 PASS not_null_fct_orders_customer_id ................................... [PASS in 0.13s]
09:46:49  8 of 15 START test not_null_stg_orders_net_amount .............................. [RUN]
09:46:49  6 of 15 PASS not_null_fct_orders_product_category .............................. [PASS in 0.11s]
09:46:49  7 of 15 PASS not_null_mart_daily_sales_order_date .............................. [PASS in 0.09s]
09:46:49  5 of 15 PASS not_null_fct_orders_order_id ...................................... [PASS in 0.12s]
09:46:49  9 of 15 START test not_null_stg_orders_order_id ................................ [RUN]
09:46:49  10 of 15 START test relationships_fct_orders_customer_id__customer_id__ref_dim_customer_  [RUN]
09:46:49  11 of 15 START test relationships_fct_orders_product_category__product_category__ref_dim_product_category_  [RUN]
09:46:49  8 of 15 PASS not_null_stg_orders_net_amount .................................... [PASS in 0.16s]
09:46:49  12 of 15 START test unique_dim_customer_customer_id ............................ [RUN]
09:46:49  9 of 15 PASS not_null_stg_orders_order_id ...................................... [PASS in 0.17s]
09:46:49  13 of 15 START test unique_dim_product_category_product_category ............... [RUN]
09:46:49  12 of 15 PASS unique_dim_customer_customer_id ................................... [PASS in 0.10s]
09:46:49  11 of 15 PASS relationships_fct_orders_product_category__product_category__ref_dim_product_category_  [PASS in 0.15s]
09:46:49  10 of 15 PASS relationships_fct_orders_customer_id__customer_id__ref_dim_customer_  [PASS in 0.21s]
09:46:49  14 of 15 START test unique_fct_orders_order_id ................................. [RUN]
09:46:49  15 of 15 START test unique_stg_orders_order_id ................................. [RUN]
09:46:49  13 of 15 PASS unique_dim_product_category_product_category ..................... [PASS in 0.06s]
09:46:49  14 of 15 PASS unique_fct_orders_order_id ....................................... [PASS in 0.06s]
09:46:49  15 of 15 PASS unique_stg_orders_order_id ....................................... [PASS in 0.08s]
09:46:49
09:46:49  Finished running 15 data tests in 0 hours 0 minutes and 0.83 seconds (0.83s).
09:46:49
09:46:49  Completed successfully
09:46:49
09:46:49  Done. PASS=15 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=15
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project> dbt docs generate
09:46:57  Running with dbt=1.12.6
09:46:57  Registered adapter: duckdb=1.11.0
09:46:57  Found 5 models, 15 data tests, 1 source, 500 macros
09:46:57
09:46:57  Concurrency: 4 threads (target='dev')
09:46:57
09:46:58  Building catalog
09:46:58  Catalog written to D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project\target\catalog.json
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject\dbt_project>
```

Model	Materialisasi	Jumlah Baris	Jenis Test yang Lolos
stg_orders	view	30	unique, not_null
dim_customer	table	25	unique, not_null
dim_product_category	table	4	unique, not_null
fct_orders	table	30	unique, not_null, relationships, accepted_values
mart_daily_sales	table	30	not_null

Perintah dbt debug memverifikasi profil, berkas proyek, dan koneksi ke warehouse dengan hasil All checks passed. Perintah dbt run membangun lima model, yaitu satu view dan empat table, dalam 0,74 detik dengan PASS=5, lalu dbt test menjalankan 15 pengujian dengan PASS=15 tanpa error. Terakhir, dbt docs generate menulis katalog dokumentasi ke folder target.

G. Dokumentasi dan lineage



Lineage graph memperlihatkan alur raw.orders menuju stg_orders, lalu bercabang ke dim_customer, dim_product_category, dan fct_orders, sebelum fct_orders diteruskan ke mart_daily_sales. Dokumentasi dan lineage memudahkan pemeliharaan pipeline karena dampak perubahan pada satu model terhadap model hilirnya dapat dilihat sebelum perubahan dijalankan.

H. Konsep yang wajib dijelaskan

Star schema menghubungkan fact langsung ke dimensi yang denormalisasi sehingga join lebih sedikit dan query lebih sederhana, dengan konsekuensi sebagian atribut berulang. Snowflake schema memecah dimensi menjadi beberapa tabel yang lebih ternormalisasi, sehingga redundansi berkurang tetapi jumlah join bertambah. Grain adalah makna satu baris pada tabel

fakta dan harus ditentukan sebelum fact dibuat, karena grain menentukan kunci, agregasi yang sah, dan cara join. Incremental model dibutuhkan ketika data sumber jauh lebih besar dibanding data baru per periode, dan penerapannya memerlukan unique key, filter watermark dengan lookback, serta strategi merge agar data yang datang terlambat tidak terlewat.

Tahap 6: Jawaban Pertanyaan Analitik

7. Mini Project > data_processing_miniproject > scripts > run_queries.py > ...

```python
"""Menjalankan kelima pertanyaan analitik wajib dan mencetak hasilnya."""
import argparse

import duckdb

PERTANYAAN = [
    ("1. Nilai penjualan bersih per hari", """
        SELECT order_date, ROUND(SUM(net_sales), 2) AS net_sales
        FROM mart_daily_sales
        GROUP BY order_date
        ORDER BY order_date
    """),
    ("2. Kategori produk dengan penjualan bersih tertinggi", """
        SELECT product_category, ROUND(SUM(net_sales), 2) AS net_sales
        FROM mart_daily_sales
        GROUP BY product_category
        ORDER BY net_sales DESC
    """),
    ("3. Jumlah dan nilai pesanan menurut status", """
        SELECT status,
               COUNT(*) AS order_count,
               ROUND(SUM(net_amount), 2) AS total_net_amount
        FROM fct_orders
        GROUP BY status
        ORDER BY order_count DESC
    """),
    ("4. Metode pembayaran paling sering digunakan", """
        SELECT payment_method, COUNT(*) AS order_count
        FROM fct_orders
        GROUP BY payment_method
        ORDER BY order_count DESC
    """),
    ("5. Total penjualan bersih yang diakui (paid dan shipped)", """
        SELECT ROUND(SUM(net_sales), 2) AS net_sales_diakui
        FROM mart_daily_sales
    """),
]
```

```python
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--db", default="warehouse/warehouse.duckdb")
    args = ap.parse_args()

    # read_only agar berkas tidak terkunci bila dbt atau proses lain sedang memakainya
    con = duckdb.connect(args.db, read_only=True)
    for judul, sql in PERTANYAAN:
        print("=" * 70)
        print(judul)
        print("-" * 70)
        print(con.execute(sql).fetchdf().to_string(index=False))
        print()
    con.close()
    print("Catatan: pertanyaan nomor 5 pada brief soal record yang ditolak kontrak")
    print("dijawab dari data/rejected/rejected_orders.csv, bukan dari warehouse,")
    print("karena record cacat memang tidak pernah dimuat ke dalamnya.")


if __name__ == "__main__":
    main()
```

```
(.venv) PS D:\CODE\Data Processing Workflow\7. Mini Project\data_processing_miniproject> python scripts/run_queries.py
======================================================================
1. Nilai penjualan bersih per hari
----------------------------------------------------------------------
order_date  net_sales
2026-09-01     363.57
2026-09-02     196.42
2026-09-03     110.00
2026-09-04     156.60
2026-09-05     275.60
2026-09-06    1022.60
2026-09-07     192.46
2026-09-08     285.48
2026-09-09     244.00
2026-09-10     565.30

======================================================================
2. Kategori produk dengan penjualan bersih tertinggi
----------------------------------------------------------------------
product_category  net_sales
     Electronics    1605.12
            Home     996.98
         Fashion     573.20
           Books     236.73

======================================================================
3. Jumlah dan nilai pesanan menurut status
----------------------------------------------------------------------
   status  order_count  total_net_amount
     paid           20           2443.85
  shipped            8            968.18
cancelled            1             62.40
 refunded            1            474.05

======================================================================
4. Metode pembayaran paling sering digunakan
----------------------------------------------------------------------
payment_method  order_count
          card           14
      transfer            8
       ewallet            8

======================================================================
5. Total penjualan bersih yang diakui (paid dan shipped)
----------------------------------------------------------------------
 net_sales_diakui
          3412.02

Catatan: pertanyaan nomor 5 pada brief soal record yang ditolak kontrak
dijawab dari data/rejected/rejected_orders.csv, bukan dari warehouse,
karena record cacat memang tidak pernah dimuat ke dalamnya.
```

A. Berapa nilai penjualan bersih per hari?

Penjualan bersih tercatat pada sepuluh tanggal, dari 1 hingga 10 September 2026, dengan tanggal dihitung dalam UTC sesuai timestamp sumber. Penjualan tertinggi terjadi pada 6 September 2026 sebesar 1.022,60 dan terendah pada 3 September 2026 sebesar 110,00, dengan total keseluruhan 3.412,02. Bila tanggal diturunkan dengan zona waktu WIB, order O1018 dan O1024 bergeser sehari, sehingga normalisasi ke UTC membuat hasil ini sama di mesin mana pun.

B. Kategori produk mana yang memiliki penjualan bersih tertinggi?

Electronics menjadi kategori dengan penjualan bersih tertinggi sebesar 1.605,12, diikuti Home 996,98, Fashion 573,20, dan Books 236,73. Electronics menyumbang hampir separuh dari total penjualan bersih yang diakui.

C. Bagaimana jumlah dan nilai pesanan menurut status?

Pesanan didominasi status paid sebanyak 20 pesanan senilai 2.443,85, disusul shipped 8 pesanan senilai 968,18, sedangkan refunded dan cancelled masing-masing satu pesanan senilai 474,05 dan 62,40. Nilai refunded dan cancelled tidak dihitung sebagai penjualan yang diakui, sehingga total penjualan bersih hanya berasal dari paid dan shipped.

D. Metode pembayaran mana yang paling sering digunakan?

Card merupakan metode pembayaran yang paling sering digunakan dengan 14 pesanan, sedangkan ewallet dan transfer sama-sama digunakan pada 8 pesanan.

E. Berapa record yang ditolak kontrak, dan apa penyebab terbesarnya?

Sebanyak 6 dari 36 record atau 16,67 persen ditolak oleh data contract, sebagaimana terlihat pada hasil validasi di Tahap 2. Keenam record tersebut masing-masing melanggar satu aturan yang berbeda, yaitu quantity nol, unit_price negatif, discount_pct di atas 1, order_ts tidak valid, status pending, dan customer_id kosong, sehingga tidak ada satu penyebab yang mendominasi. Seluruh angka penjualan pada tahap ini memakai aturan bisnis bahwa hanya status paid dan shipped yang diakui sebagai penjualan.

Tahap 7: Penutup

A. Ringkasan keputusan desain

Aspek	Keputusan	Alasan Singkat
Ukuran halaman	10 record per halaman	Halaman cukup kecil agar pagination dan retry per halaman terlihat jelas
Kolom watermark	order_ts, ditambah deduplikasi order_id	Timestamp memberi batas inkremental; dedup menjaga idempotensi bila timestamp rusak
Mode validasi kontrak	Karantina per-record, opsi fail-fast	Record sah tetap diproses, record cacat tersimpan dengan alasannya
Jumlah partisi dan message key	3 partisi, key order_id	Paralelisme hingga tiga consumer dan urutan per order terjaga
Strategi commit dan delivery semantics	Commit manual setelah sink, at-least-once	Tidak ada pesan hilang; duplikasi dinetralkan upsert di sink
Desain dimensional	Star schema, fct grain per order_id	Join sedikit dan query analitik sederhana
Aturan penjualan diakui	Hanya paid dan shipped	Cancelled dan refunded bukan pendapatan yang diakui
Transformasi	Polars lazy, gabung batch dan streaming, UTC	Satu tabel bersih tanpa duplikat; tanggal tidak bergantung zona waktu mesin

B. Keterbatasan

Pipeline ini memiliki beberapa keterbatasan. Sumber data berupa berkas CSV lokal yang disimulasikan sebagai REST API, sehingga penanganan autentikasi, status HTTP, dan header Retry-After pada API sungguhan belum teruji langsung. Jaminan pengiriman yang dicapai adalah at-least-once dengan idempotensi di sink, dan exactly-once end-to-end tidak diklaim. Benchmark dijalankan pada dataset yang diperbesar secara sintetis di satu mesin, sehingga angkanya bergantung pada spesifikasi perangkat dan tidak mewakili kinerja klaster terdistribusi. Dataset asli hanya berisi 36 baris sehingga belum menggambarkan skala produksi. Selain itu, klien kafka-python 3.0.11 memunculkan peringatan deprecation karena serializer berbasis fungsi lambda belum memakai antarmuka Serializer yang baru, meskipun tidak memengaruhi hasil.

C. Pembagian pekerjaan

Mini project ini dikerjakan secara individu, sehingga seluruh tahap mulai dari ingestion, validasi kontrak, streaming, transformasi, hingga pemodelan dbt dan penyusunan laporan dikerjakan sendiri. Asisten AI digunakan sebagai alat bantu untuk menyusun panduan dan kerangka kode, sedangkan seluruh perintah dijalankan sendiri.